Project Blueprint: Terminal to Web Ecosystem

Transitioning from Codewars Logic to Full-Stack Python Application Development

Since your development workflow (IDE, Terminal, Virtual Environments, and Git) is completely locked down, you are in an ideal position to build a modern system. This plan replaces standard desktop GUI tools with a **modern decoupled web architecture**. You will build a backend API using Python and wire it up to a frontend user interface using standard web technologies.

The Core Application Architecture

Instead of single-script execution, your project will run on a **Client-Server Architecture**:

- **The Backend (Python/FastAPI):** Acts as the system engine. It communicates with your database, handles raw data manipulation, executes calculation logic, and exposes endpoints (URLs) that transmit clean JSON data.
- **The Frontend (HTML/CSS/JavaScript):** Acts as the viewport. It runs entirely within the web browser, handles user clicks, captures inputs, sends background web requests to Python, and dynamically renders data onto the screen without forcing manual page refreshes.

Phase 1: Database Engine & Local Logic

PHASE 1

Structured Storage & Core Methods

Before launching web servers, you must construct a rock-solid, local Python layer that abstractly handles data persistence.

- **Objective:** Design a lightweight relational data storage model for a personal tracking asset (e.g., an Expense/Asset Manager or a Metrics Logger).
- **The Stack:** `Python 3.x` `sqlite3`
- **Key Tasks:**
  - Write an initial schema configuration script to instantiate a local SQLite file (`app.db`) containing structured tables with explicit constraints (Primary Keys, Foreign Keys, Datatypes).
  - Construct an isolated Python module focused entirely on database CRUD execution. Wrap raw SQL queries directly inside clean Python functions (e.g., `add_record(amount, category)`, `get_all_records()`).
  - Validate this module via your local terminal terminal before proceeding. Ensure data safely persists on disk even after your script completely closes down.

Phase 2: Developing the Web API Backend

PHASE 2

Exposing Logic over HTTP via FastAPI

In this phase, you transform your native terminal functions into an active server application listening to web communication protocols.

- **Objective:** Build a high-performance REST API wrapper around your database module.
- **The Stack:** `FastAPI` `Uvicorn` `Pydantic`
- **Key Tasks:**
  - Initialize an asynchronous FastAPI instance and configure Uvicorn to run as a live local reload server.
  - Utilize Pydantic data schemas to declare rigid validation parameters for incoming user request payloads.
  - Map web access routes directly into your custom database functions:
    - `GET /api/records` → Returns a raw list of objects processed straight out of SQLite.
    - `POST /api/records` → Receives an external incoming payload and writes it directly to disk.
  - Enable Cross-Origin Resource Sharing (CORS) middleware parameters so your browser security settings do not explicitly block local script exchanges down the line.

Phase 3: Building the Visual Web UI

PHASE 3

The Client Interface Layer

Now you shift focus out of your Python ecosystem directly into native browser execution environments to give your product an accessible visual layout.

- **Objective:** Build a single-page web view utilizing standard document structures and dynamic asynchronous styling tools.
- **The Stack:** `HTML5` `CSS3 (Modern Flexbox)` `JavaScript (ES6 Fetch API)`
- **Key Tasks:**
  - Draft an index layout containing data input elements, processing submission buttons, and structured layout tables.
  - Write native JavaScript functions utilizing the `fetch()` API mechanism to seamlessly request data streams out of your running Python application without manual screen updates.
  - Loop over your JSON response payloads and programmatically construct rows straight inside your HTML document tables.
  - Attach standard event listeners onto your visual forms to intercept browser events, translate elements into raw payloads, and transfer them upstream via post requests back into your database layers.

The Recommended Target Folder Blueprint

Maintain an ordered, isolated environment right from the start. Structure your target directories using this specific multi-file convention:

```
my_web_project/
├── app.db              # Local SQLite database file (auto-generated)
├── database.py         # Pure Python database operations (CRUD logic)
├── main.py             # FastAPI server initiation and routing handles
└── static/             # Visual client assets directory
    ├── index.html      # Visual markup document structure
    ├── style.css       # Clean interface layout formatting definitions
    └── app.js          # Dynamic Fetch engine and document mutations
```

Why This Bridges the Gap to Professional Work

By bypassing basic GUI wrappers (like Tkinter) and stepping straight into functional Web Frameworks, you instantly replicate production-grade environments. Handling multi-origin file requests, interpreting network data streams, and managing structured storage schemas constitutes the exact foundational matrix expected across standard software engineering and real-world development applications.